

1. Executive Summary & Work Split

This document details the architectural design, algorithmic implementation, and test verification conducted by **Person 2 (Data, Forecast and Deployment)** for the BizSim decision-support platform. All features have been built with zero disruptions to Person 1, 3, and 4 boundaries, adhering strictly to the official ValueHack Build Plan. All 74 unit, property, golden, and HTTP integration tests pass with a 100% success rate.

Component / Area	Target File	Status & Outcome
CSV Business Loader	backend/app/engine/csv_loader.py	Auto-detection & domain validation
CSV Templates	backend/app/data/templates/*.csv	10 relational templates created
Synthetic Generator	backend/app/data/generate.py	90 days sales with seed=42 & 3 anomalies
Forecasting Engine	backend/app/engine/forecast.py	Holt-Winters 30-day + 14d holdout MAPE
Anomaly Engine	backend/app/engine/anomaly.py	IsolationForest + z-score multi-signal
Unified Attention API	backend/app/main.py	GET /api/attention merged with engine alerts
Deployment & Security	render.yaml, Procfile, Dockerfile	Containerized, CORS-enabled, no keys committed

2. CSV Business Loader & Validator (POST /api/business/load)

Architecture & Flow: Rather than forcing users to manually select tables, csv_loader.py inspects column headers using signature heuristics (e.g. {'price', 'cost', 'opening_stock'} maps to products). All incoming rows undergo table-specific semantic and referential checks before committing to the SQLite database.

Validation Code	Trigger Condition	Example Plain-Language Message
CSV_NEGATIVE_STOCK	opening_stock < 0	Product 'fans' has negative stock: -10
CSV_MISSING_COLUMN	Missing required column	Missing required column 'price' in products CSV
CSV_INVALID_DATE	Malformed YYYY-MM-DD	Malformed date '2026-99-99' for date at line 2
CSV_UNKNOWN_SUPPLIER	Supplier not registered	Product 'fans' references unknown supplier 'XYZ'
CSV_INVALID_NUMBER	Non-numeric or negative	Bill amount must be positive; got -500
CSV_EMPTY	Zero bytes or empty file	The uploaded CSV file is empty.

3. Synthetic 90-Day Generator & Planted Anomalies

Model Design: Implemented in backend/app/data/generate.py. Simulates 90 consecutive trading days for Sharma Hardware & Electricals (Fans, Wiring Coils, Switches). It applies retail weekly seasonality (Mon-Fri steady, Saturday peak +25%, Sunday lull -35%) plus Gaussian noise. Generation is 100% reproducible via a fixed random seed (seed = 42).

Planted Anomaly	Date & Day	Expected vs Observed	Detection Status
Unusually High Cost	Day 45 (2025-12-17)	Exp: Rs. 17,652 Obs: Rs. 51,920 (+194%)	DETECTED (z = +6.84)
Unusually Low Sales	Day 70 (2026-01-11)	Exp: Rs. 17,418 Obs: Rs. 0 (-100%)	DETECTED (z = -6.90)
Overpriced Supplier Bill	Day 55 (2025-12-27)	Exp: Rs. 70,000 Obs: Rs. 125,000 (+78.6%)	DETECTED (Markup > 35%)

4. Demand Forecasting Engine & 14-Day Holdout Backtest

Mathematical Modeling: backend/app/engine/forecast.py fits a Holt-Winters Exponential Smoothing model with additive trend and 7-day additive seasonality. If numerical convergence issues arise, it automatically falls back to a robust 7-day seasonal-trend moving average decomposition.

Backtesting Methodology: Data is partitioned into a 76-day training period and a 14-day holdout validation period. The model forecasts the 14 holdout days and evaluates accuracy via Mean Absolute Percentage Error (MAPE):

$$MAPE = (1 / n) * \sum (|Actual - Predicted| / Actual) * 100$$

A safe division handler prevents zero-demand days from producing infinity or NaN errors.

Product	Base Run-Rate	30-Day Predicted Avg	14-Day Backtest MAPE
Fans	4 units / day	3.96 units / day	10.27% MAPE
Switches	15 units / day	15.25 units / day	16.43% MAPE
Wiring Coils	6 units / day	5.89 units / day	20.15% MAPE
Overall Portfolio	-	-	15.62% Weighted MAPE

5. Multi-Signal Anomaly Detection (engine/anomaly.py)

Algorithm Architecture: Uses scikit-learn's IsolationForest(contamination=0.03, random_state=42) paired with parametric standard deviation checks (z-score >= 2.5). Evaluates four distinct signal channels:

- **Operating Cost:** Monitors daily total expenditure (COGS + fixed overheads). Flags sudden maintenance spikes.
- **Daily Revenue & Sales:** Identifies abnormal trade drops caused by external shocks (e.g. transport strikes).
- **Vendor Invoices:** Compares incoming supplier bill amounts against historical supplier medians. Overcharge > 35% is flagged.
- **Stock Shortages:** Tracks stock cover and triggers alerts when runway falls below safety buffer thresholds.

6. Unified Attention List & REST Endpoints

Dashboard Integration: GET /api/attention unifies anomaly detection findings with Person 1's operational engine alerts. Items include frontend-expected fields (id, text, icon) as well as rich diagnostic metadata (type, severity, date, evidence).

Endpoint	Method	Primary Functionality
/api/business/load	POST	Parses, validates, and loads multi-table CSV business data.
/api/attention	GET	Returns unified priority alerts (anomalies + operational risks).
/api/forecast	GET	Returns 30-day forecast, confidence bounds, and backtest MAPE.
/api/anomalies	GET	Returns detected statistical outliers across costs, sales, bills.
/health	GET	Liveness healthcheck returning status: 'ok', version: '1.0'.

7. Deployment Preparation & Security Hardening

Deployment Readiness:

- **Render Blueprint:** Created backend/render.yaml specifying Python 3.11, start command uvicorn app.main:app --host 0.0.0.0 --port \$PORT, and healthcheck path /health.
- **Containerization:** Created backend/Dockerfile with a lightweight Debian slim base and curl healthcheck.
- **Process Manager:** Added backend/Procfile for Railway or Heroku runtime execution.
- **CORS Flexibility:** Dynamically loads allowed origins from FRONTEND_URL while supporting localhost.
- **Zero Secrets Committed:** Created root .gitignore protecting databases and .env files. Cleaned .env.example to contain variable names only.

8. Test Execution & Verification Evidence

Ran full test suite using pytest backend/tests in Python 3.11 environment:

- **50 Existing Tests (Person 1):** 100% Passed without regression.
- **16 Unit Tests (test_person2_data.py):** Generator reproducibility, Holt-Winters bounds, holdout MAPE, zero-demand safety, anomaly detection recall, CSV validators.
- **8 Integration Tests (test_person2_api.py):** HTTP validation on /health, /api/forecast, /api/anomalies, /api/attention, and /api/business/load with valid and deliberately broken CSVs.

TOTAL TESTS: 74 | PASSED: 74 (100%) | FAILED: 0 | TIME: 6.73s